

Version: 1.0



Selection

C-Caractères-Arithmétique

Résumé

Ce projet se concentre sur la manipulation de nombres et les opérations arithmétiques en langage C.

#C

#Arithmétique

#ManipulationDeNombres

42

Chapitre 3

Exercice 0 : ft_putnbr

	Exercice: 0	
ft_putnbr		
Répertoire: ex0/		
Fichiers à Rendre: ft_putnbr.c		
Autorisé: write		

- Écrivez une fonction qui affiche le nombre passé en paramètre. La fonction doit pouvoir afficher toutes les valeurs possibles d'une variable de type `int`.

Prototype :

```
void ft_putnbr(int nb);
```

- Par exemple :

ft_putnbr(42) affiche :

```
?> ./a.out
42
?>
```

Chapitre 4

Exercice 1 : ft_print_comb

	Exercice: 1	
ft_print_comb		
Répertoire: ex1/		
Fichiers à Rendre: ft_print_comb.c		
Autorisé: write		

- Écrivez une fonction qui affiche toutes les combinaisons possibles de trois chiffres différents dans l'ordre croissant, listées elles-mêmes par ordre croissant — oui, la répétition est volontaire.
- Voici ce qu'on attend comme sortie :

Exemple :

```
>./a.out | cat -e  
012, 013, 014, 015, 016, 017, 018, 019, 023, ..., 789$>
```

- 987 n'apparaît pas car 789 l'a déjà été.
- 999 n'apparaît pas car le chiffre 9 est utilisé plus d'une fois.

Prototype :

```
void ft_print_comb(void);
```



As-tu vérifié avec ton voisin de droite ?

Chapitre 5

Exercice 2 : ft_print_comb2

	Exercice: 2	
ft_print_comb2		
Répertoire: ex2/		
Fichiers à Rendre: ft_print_comb2.c		
Autorisé: write		

- Écrivez une fonction qui affiche toutes les combinaisons possibles de deux nombres à deux chiffres (XX XX) entre 00 et 99, listées par ordre croissant.
- Voici ce qu'on attend comme sortie :

Exemple :

```
>./a.out | cat -e  
00 01, 00 02, 00 03, ..., 00 99, 01 02, ..., 97 99, 98 99$>
```

Prototype :

```
void ft_print_comb2(void);
```



Inspire-toi des autres, mais ne les laisse pas faire à ta place.

Chapitre 6

Soumission et évaluation par les pairs

Soumettez votre devoir dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué pendant la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous ne devez rendre que les fichiers demandés par le sujet de ce projet.

Chapitre VII

Exercice 04 : ft_putnbr_base

	Exercice : 04
ft_putnbr_base	
Dossier de rendu : <i>ex04/</i>	
Fichiers à rendre : <code>ft_putnbr_base.c</code>	
Fonctions Autorisées : <code>write</code>	

- Écrire une fonction qui affiche un nombre dans le terminal dans une base donnée.
- Ce nombre est fourni sous la forme d'un `int` et la base sous la forme d'une chaîne de caractères.
- La base contient tous les symboles utilisables pour afficher le nombre :
 - 0123456789 est la base couramment utilisée pour représenter nos nombres décimaux;
 - 01 est une base binaire;
 - 0123456789ABCDEF est une base hexadecimale;
 - poneyvif est une base octale.
- La fonction doit gérer les nombres négatifs.
- Si un paramètre contient une erreur la fonction n'affiche rien. Une erreur peut être :
 - base est vide ou est de taille 1;
 - base contient deux fois le même caractère;
 - base contient les caractères + ou -.
- Elle devra être prototypée de la façon suivante :

```
void ft_putnbr_base(int nbr, char *base);
```

Chapitre VIII

Exercice 05 : ft_atoi_base

	Exercice : 05
	ft_atoi_base
	Dossier de rendu : ex05/
	Fichiers à rendre : ft_atoi_base.c
	Fonctions Autorisées : Aucune

- Ecrire une fonction qui convertit le début de la chaîne pointée par str en entier de type int.
- str est dans une base spécifique passée en second argument de la fonction.
- A part le système de base, cette fonction doit reproduire le comportement de ft_atoi.
- Si un paramètre contient une erreur la fonction renvoie 0. Une erreur peut être :
 - la base est vide ou est de taille 1 ;
 - la base contient deux fois le même caractère ;
 - la base contient les caractères + ou - ou des whitespaces ;
- Elle devra être prototypée de la façon suivante :

```
int ft_atoi_base(char *str, char *base);
```